

MAST32001 Computational Statistics - Lecture 3

Random number generation

Klaus Nordhausen

University of Helsinki

WS 2026

Today

- ▶ What do we mean by random numbers?
- ▶ Physical and pseudo-random number generation
- ▶ Uniform generators and linear congruential generators
- ▶ From $U(0, 1)$ to other distributions
- ▶ Relationships between common distributions
- ▶ Inverse, transformation, and acceptance-rejection methods
- ▶ Generating normal random variables

Randomness in computation

Random sample

A random sample $X_1, \dots, X_n$ from a distribution F consists of random variables that are

$$X_1, \dots, X_n \stackrel{iid}{\sim} F.$$

For simulation we want numbers that behave like realizations of such variables.

Two conceptually different routes are possible:

1. obtain randomness from a physical process;
2. generate a deterministic sequence designed to behave statistically like randomness.

Using physical processes

Physical randomness can come from processes whose outcomes cannot be predicted exactly:

- ▶ tossing coins or dice;
- ▶ drawing shuffled cards;
- ▶ radioactive decay;
- ▶ thermal or electronic noise;
- ▶ photon arrival times;
- ▶ atmospheric noise.

The physical observation is usually converted into bits and then into integers or $U(0, 1)$ values.

From a physical measurement to random bits

Suppose a device records a noisy physical quantity Y .

A naive rule such as

$$B = 1\{Y > 0\}$$

does not automatically give a fair random bit: the device may be biased or observations may be dependent.

A real random-number system therefore typically needs

physical entropy $\rightarrow$ measurement $\rightarrow$ debiasing / extraction $\rightarrow$ random bits.

Physical randomness is not the same thing as simply measuring a noisy signal.

A simple debiasing idea: von Neumann extractor

Suppose independent bits satisfy

$$P(B = 1) = p, \quad P(B = 0) = 1 - p,$$

where p is unknown.

Read bits in pairs:

$$01 \mapsto 0, \quad 10 \mapsto 1,$$

and discard 00 and 11.

Why is the output unbiased?

$$P(01) = (1 - p)p = P(10).$$

This removes marginal bias under the independence assumption, but it can discard many observations.

Advantages and disadvantages of physical randomness

Advantages:

- ▶ nondeterministic source;
- ▶ useful when unpredictability itself is essential.

Disadvantages for statistical simulation:

- ▶ difficult to reproduce;
- ▶ may be slow or require hardware/network access;
- ▶ physical sources can be biased or dependent;
- ▶ data collection and extraction must be validated;
- ▶ debugging and exact replication become harder.

For computational statistics, **reproducibility is usually a reason to prefer pseudo-randomness.**

Real random numbers from external services

Some services provide random bits generated from physical processes such as atmospheric or quantum noise.

These can be accessed from R through network interfaces or packages, but they introduce external dependencies.

For this course, the important distinction is:

physical randomness: nondeterministic

PRNG: deterministic but reproducible

We will use pseudo-random generators for statistical simulation.

Real random numbers directly from R

The CRAN package `random` provides an R interface to RANDOM.ORG.

RANDOM.ORG obtains randomness from **atmospheric noise** rather than from an R pseudo-random number generator.

```
# install.packages("random")  
library(random)
```

```
randomNumbers(  
  n = 10,  
  min = 1,  
  max = 100,  
  col = 1  
)
```

Unlike `runif()` or `sample()`, these numbers are retrieved from an external true-random-number service.

Therefore this example requires an internet connection.

Real randomness is not reproducible

```
set.seed(20260812)
sample(1:100, 4)
```

```
## [1] 37 18 12 82
```

```
set.seed(20260812)
sample(1:100, 4)
```

```
## [1] 37 18 12 82
```

```
library(random)
set.seed(20260812)
t(randomNumbers(4, min = 1, max = 100, col = 1))
```

```
##      [,1] [,2] [,3] [,4]
## V1      67    71    89    87
```

```
set.seed(20260812)
t(randomNumbers(4, min = 1, max = 100, col = 1))
```

```
##      [,1] [,2] [,3] [,4]
## V1      77    52     5    87
```

Real randomness is not reproducible

The first two calls reproduce the same pseudo-random sample.

The calls to RANDOM.ORG request new physical randomness. `set.seed()` does not control them.

This is exactly why **true randomness is usually not desirable for reproducible statistical simulation.**

Pseudo-random numbers

Pseudo-random number generator

A pseudo-random number generator (PRNG) has an internal state S_n and deterministic update rule

$$S_{n+1} = T(S_n).$$

An output function produces

$$U_n = G(S_n).$$

The initial state is determined by a **seed**.

Thus

$$\text{seed} \longrightarrow S_0 \longrightarrow S_1 \longrightarrow S_2 \longrightarrow \cdots$$

is completely deterministic.

Reproducibility

```
set.seed(20260812)  
runif(5)
```

```
## [1] 0.9313843 0.8892288 0.1994912 0.2833731 0.3567195
```

```
set.seed(20260812)  
runif(5)
```

```
## [1] 0.9313843 0.8892288 0.1994912 0.2833731 0.3567195
```

The same seed and RNG algorithm reproduce the same stream.

In a simulation study, the seed is therefore part of the computational specification.

Why $U(0, 1)$ is fundamental

A PRNG normally produces numbers that behave approximately as

$$U_1, U_2, \dots \stackrel{iid}{\sim} U(0, 1).$$

Once uniform random numbers are available, we can construct many other random variables.

The central computational question becomes

$$\boxed{U(0, 1) \longrightarrow \text{target distribution}}.$$

Linear congruential generators

Congruential random number generation

A classical generator is

$$X_{n+1} = (aX_n + c) \bmod m,$$

where

- ▶ m is the modulus,
- ▶ a the multiplier,
- ▶ c the increment,
- ▶ X_0 the seed.

The corresponding uniform value is

$$U_{n+1} = \frac{X_{n+1}}{m}.$$

Work through a tiny LCG

Take

$$m = 9, \quad a = 4, \quad c = 1, \quad X_0 = 3.$$

Then

$$X_1 = (4 \cdot 3 + 1) \bmod 9 = 4,$$

$$X_2 = (4 \cdot 4 + 1) \bmod 9 = 8,$$

$$X_3 = (4 \cdot 8 + 1) \bmod 9 = 6.$$

Continue the recurrence. Because only m states exist, repetition is unavoidable.

Period

For a finite state space, a deterministic sequence must eventually repeat.

The **period** is the number of states in the repeating cycle.

For an LCG the period can never exceed m .

A long period is necessary, but **not sufficient**, for a good generator: a very long deterministic sequence may still have undesirable dependence.

LCG in R

```
lcg <- function(n, seed, a, c, m) {  
  x <- numeric(n)  
  x[1] <- seed  
  for (i in 2:n) {  
    x[i] <- (a * x[i - 1] + c) %% m  
  }  
  x / m  
}
```

```
lcg(15, seed = 3, a = 4, c = 1, m = 9)
```

```
## [1] 0.3333333 0.4444444 0.8888889 0.6666667 0.7777778 0.2222222 0.0000000  
## [8] 0.1111111 0.5555556 0.3333333 0.4444444 0.8888889 0.6666667 0.7777778  
## [15] 0.2222222
```

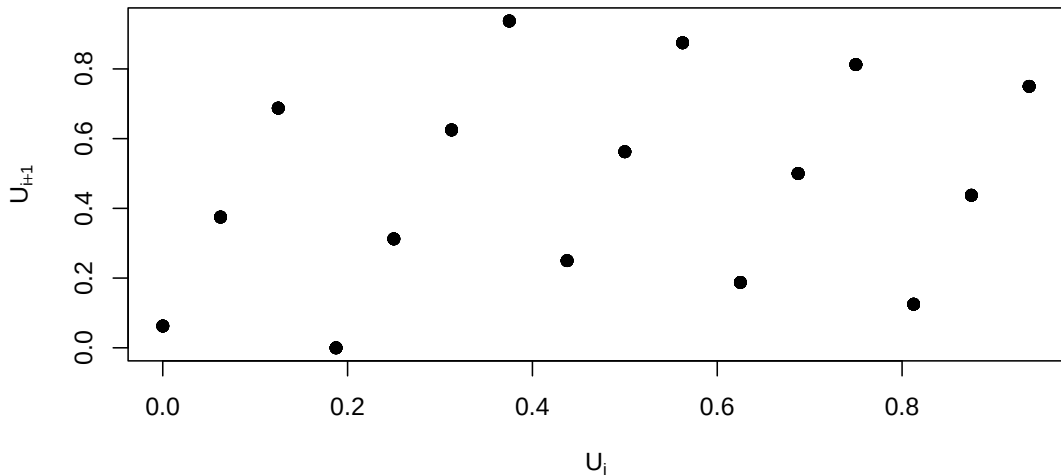
Successive values reveal structure

```
u_bad <- lcg(200, seed = 1, a = 5, c = 1, m = 16)

plot(u_bad[-length(u_bad)], u_bad[-1],
     xlab = expression(U[i]),
     ylab = expression(U[i+1]),
     pch = 19,
     main = "Successive values from a small LCG")
```

Successive values reveal structure

Successive values from a small LCG



A histogram can look plausible while pairs or higher-dimensional tuples reveal deterministic

What should a good PRNG provide?

We want:

- ▶ approximately uniform marginal behaviour;
- ▶ weak dependence;
- ▶ good behaviour of tuples

$$(U_i, U_{i+1}, \dots, U_{i+d-1});$$

- ▶ very long period;
- ▶ computational speed;
- ▶ reproducibility;
- ▶ reliable implementation.

For ordinary statistical work, use established software generators rather than writing an LCG yourself.

Random generators in R

```
RNGkind()
```

```
## [1] "Mersenne-Twister" "Inversion"          "Rejection"
```

```
RNGkind("Mersenne-Twister")
```

```
set.seed(20260812)
```

```
.Random.seed[1:8]
```

The exact RNG matters when reproducing simulations across software versions or implementations.

From uniforms to random variables

Probability integral transform

Let X have continuous cdf F . Then

$$F(X) \sim U(0, 1).$$

Conversely, if

$$U \sim U(0, 1),$$

then

$$X = F^{-1}(U)$$

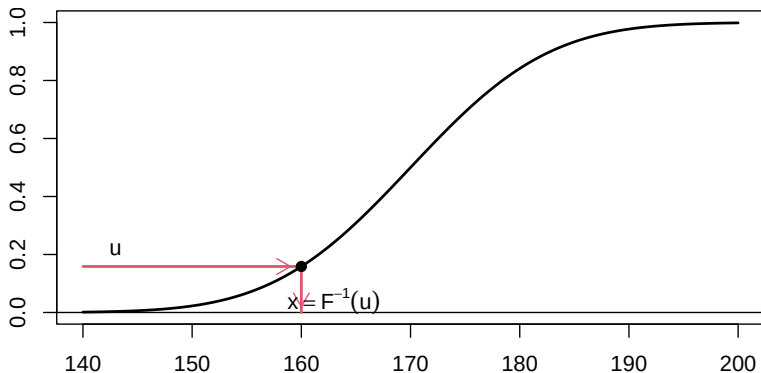
has cdf F .

This gives a general random-variable generation method.

Probability integral transform: graphical view

For a given $u \in (0, 1)$, the inverse cdf maps the horizontal probability level u to the corresponding value

$$x = F^{-1}(u).$$



Read the construction from the probability axis to the cdf and then down to the x -axis:

$$u \longrightarrow F^{-1}(u).$$

Why inverse transformation works

Let

$$X = F^{-1}(U), \quad U \sim U(0, 1).$$

For strictly increasing F ,

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) \\ &= P(U \leq F(x)) \\ &= F(x). \end{aligned}$$

Therefore X has the required distribution.

Inverse method: exponential distribution

For $X \sim \text{Exp}(\lambda)$,

$$F(x) = 1 - e^{-\lambda x}.$$

Solving $u = F(x)$,

$$x = -\frac{1}{\lambda} \log(1 - u).$$

Since $1 - U \sim U(0, 1)$,

$$\boxed{X = -\frac{\log U}{\lambda}}.$$

```
u <- runif(10000)
x <- -log(u) / 2
c(empirical = mean(x), theoretical = 1/2)
```

```
## empirical theoretical
## 0.5014223 0.5000000
```

Discrete inverse method

Suppose

$$P(X = x_j) = p_j, \quad j = 1, \dots, k.$$

Define cumulative probabilities

$$F_j = \sum_{\ell=1}^j p_{\ell}.$$

Generate $U \sim U(0, 1)$, then choose the first j satisfying

$$U \leq F_j.$$

This partitions $(0, 1)$ into intervals of lengths $p_1, \dots, p_k$.

Sampling finite populations in R

```
sample(0:1, 10, replace = TRUE)
```

```
## [1] 0 1 1 0 1 0 0 1 0 0
```

```
sample(1:49, 7, replace = FALSE)
```

```
## [1] 22 13 42 6 39 32 3
```

```
sample(LETTERS)
```

```
## [1] "K" "G" "D" "N" "T" "R" "U" "X" "H" "E" "L" "Z" "W" "M" "B" "S" "O" "P" "I"
```

```
## [20] "F" "Q" "J" "V" "C" "Y" "A"
```

```
table(sample(1:4, 1000, replace = TRUE,  
            prob = c(.2, .4, .3, .1)))
```

```
##
```

```
## 1 2 3 4
```

```
## 217 365 308 110
```


Did our generator work?

Evaluating generated random variables

After implementing a random-number generator, we should check whether its output behaves like the intended distribution.

Useful graphical checks include:

- ▶ histogram with the theoretical density;
- ▶ empirical and theoretical distribution functions;
- ▶ Q–Q plot;
- ▶ plots of successive observations to detect dependence.

These are **diagnostics**, not proofs that the generator is correct.

Histogram and theoretical density

Suppose we generated observations intended to follow $N(0, 1)$.

```
set.seed(20260812)
z_check <- rnorm(1000)

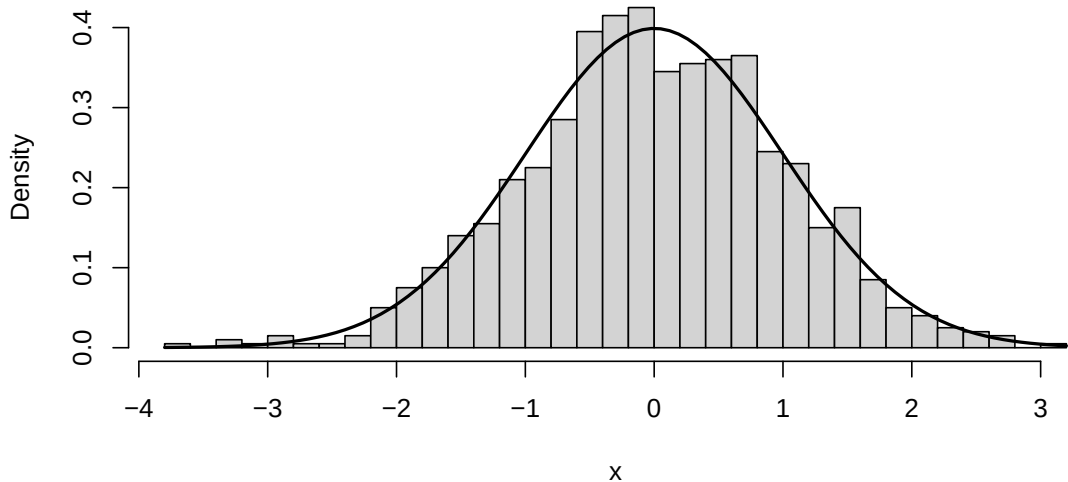
hist(z_check, probability = TRUE, breaks = 30,
     main = "Generated sample and theoretical density",
     xlab = "x")

curve(dnorm(x), add = TRUE, lwd = 2)
```

If the generator is correct, the histogram should approximately follow the target density. But histograms depend on bin choices and can hide discrepancies, especially in the tails.

Histogram and theoretical density

Generated sample and theoretical density



Q-Q plot

A Q-Q plot compares empirical quantiles

$$x_{(1)}, \dots, x_{(n)}$$

with corresponding theoretical quantiles from the target distribution.

For a normal target:

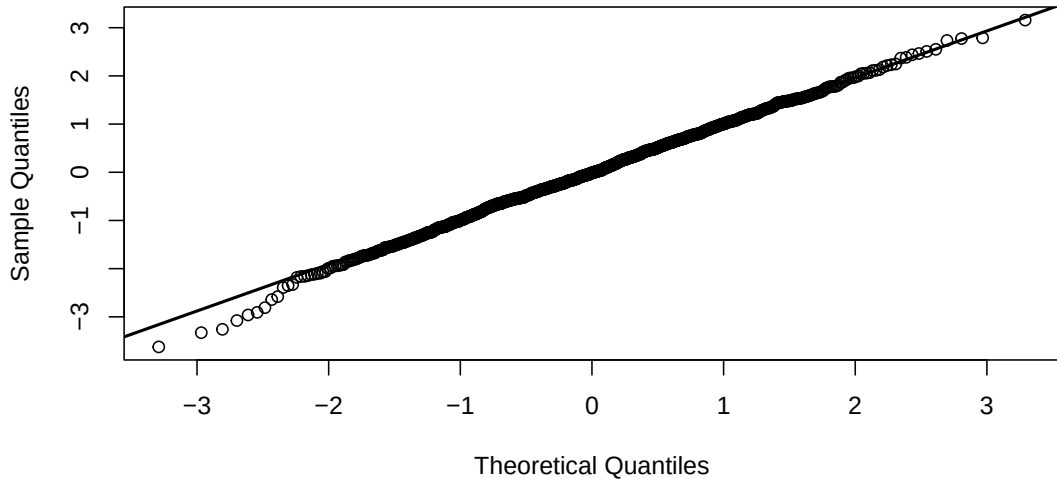
```
qqnorm(z_check,  
       main = "Normal Q-Q plot")  
qqline(z_check, lwd = 2)
```

If the target distribution is appropriate, the points should lie approximately on a straight line.

Systematic curvature indicates a distributional discrepancy.

Q-Q plot

Normal Q-Q plot



Q-Q plots for a general target distribution

For ordered observations

$$x_{(1)} \leq \cdots \leq x_{(n)},$$

compare them with

$$F^{-1}(p_i), \quad p_i \approx \frac{i - 0.5}{n}.$$

In R:

```
x_exp <- rexp(500, rate = 2)

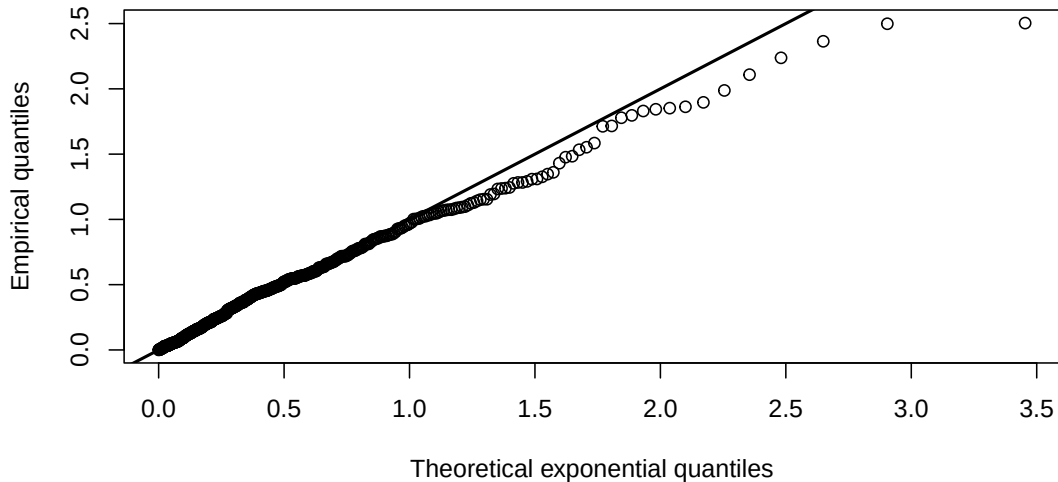
p <- ppoints(length(x_exp))
theoretical <- qexp(p, rate = 2)

plot(theoretical, sort(x_exp),
     xlab = "Theoretical exponential quantiles",
     ylab = "Empirical quantiles",
     main = "Exponential Q-Q plot")
abline(0, 1, lwd = 2)
```

So Q-Q plots are not restricted to normal distributions.

Q-Q plots for a general target distribution

Exponential Q-Q plot



Empirical distribution function

The empirical cdf is

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n 1\{X_i \leq x\}.$$

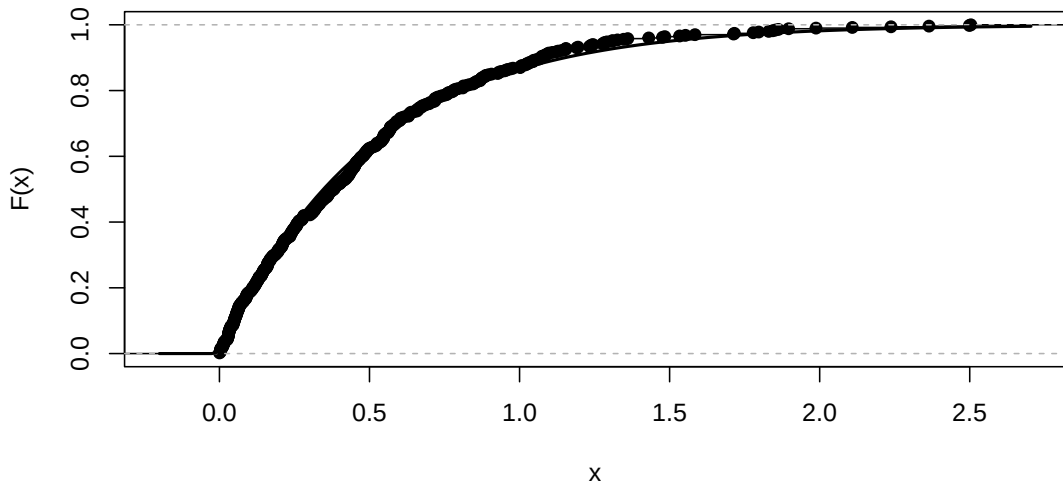
We can compare it directly with the target cdf F .

```
plot(ecdf(x_exp),  
     main = "Empirical and theoretical CDF",  
     xlab = "x", ylab = "F(x)")  
curve(pexp(x, rate = 2),  
      add = TRUE, lwd = 2)
```

This comparison uses the whole distribution rather than histogram bins.

Empirical distribution function

Empirical and theoretical CDF



Distributional fit is not enough

Suppose every U_i individually looks uniform.

We still need approximately independent observations.

A simple diagnostic is

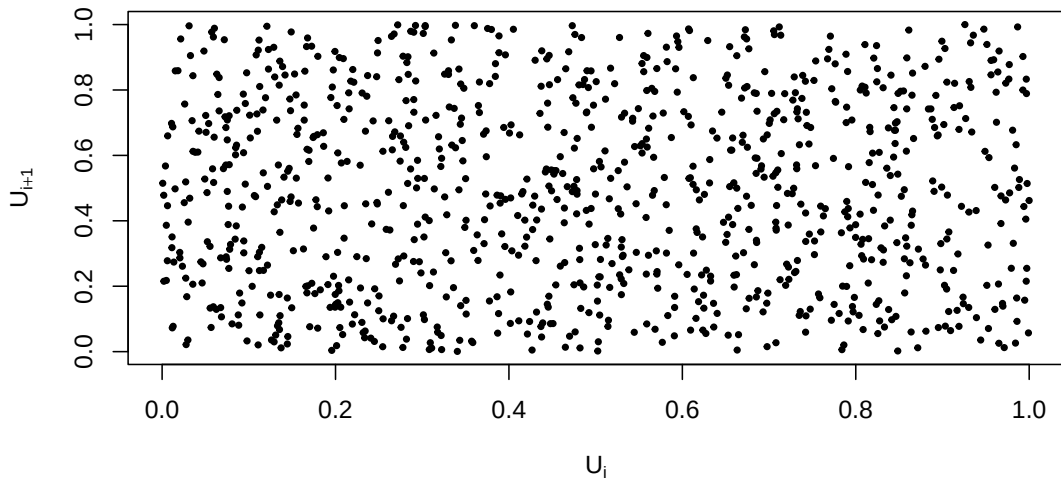
```
u_check <- runif(1000)

plot(u_check[-length(u_check)], u_check[-1],
     xlab = expression(U[i]),
     ylab = expression(U[i+1]),
     main = "Successive observations",
     pch = 19, cex = 0.5)
```

A random-number generator must get both the **marginal distribution** and the **dependence structure** right.

Distributional fit is not enough

Successive observations



What should we check?

For a newly implemented generator, a useful first checklist is:

distribution

+

dependence

For the distribution:

- ▶ histogram + theoretical density;
- ▶ Q–Q plot;
- ▶ empirical cdf + theoretical cdf;
- ▶ moments or selected quantiles.

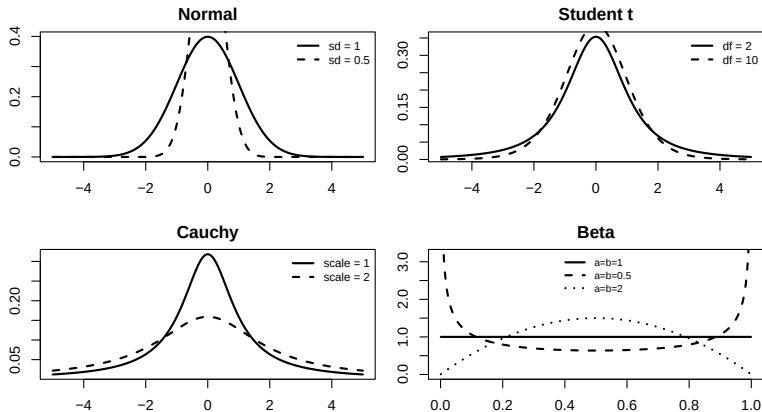
For dependence:

- ▶ successive-pair plots;
- ▶ autocorrelations;
- ▶ more specialized RNG tests when needed.

Shapes of common distributions

Continuous distributions on the real line

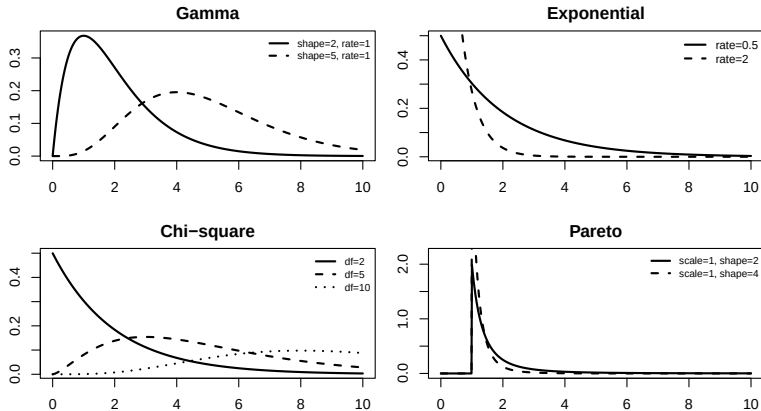
Before using distributional relationships, it is useful to remember what they look like.



Notice especially the different tail behaviour of the normal, t , and Cauchy distributions, and the very different shapes possible within the Beta family.

Positive continuous distributions

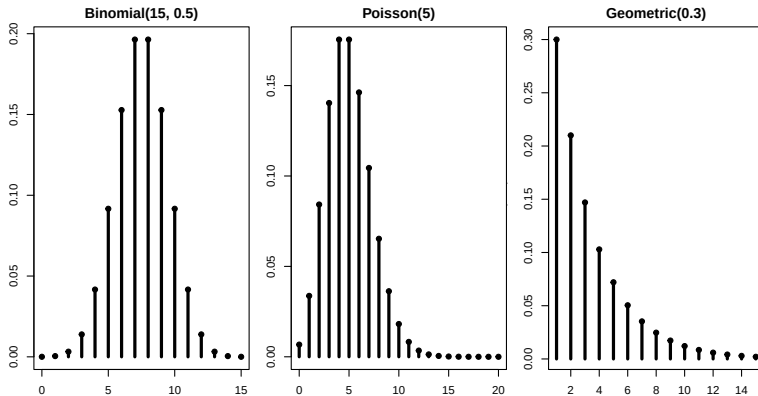
Many distributions used in simulation have support on the positive real line.



The exponential, Gamma, chi-square, and Pareto distributions all live on the positive half-line, but their shapes and especially their tails can be very different.

Some discrete distributions

For completeness, common discrete distributions also have very different shapes.



For discrete distributions we work with a **probability mass function** rather than a density.

Relationships between distributions

Why distributional relationships matter

Random-variable generation becomes easier when we exploit known identities.

Rather than inventing a new algorithm for every distribution, we can use

simple variables + transformations $\longrightarrow$ new distributions.

These relationships are also fundamental in statistical theory. A good read here is Leemis (2008)

Transformations of one variable

Some important examples:

$$X \sim N(\mu, \sigma^2) \implies e^X \sim \text{Lognormal}(\mu, \sigma^2).$$

$$X \sim \text{Exp}(\alpha) \implies Y = y_0 e^X \sim \text{Pareto}(\alpha, y_0).$$

Also,

$$X \sim N(0, 1) \implies X^2 \sim \chi_1^2.$$

Gamma, exponential and chi-square

Using the rate parameterization,

$$X \sim \text{Gamma}(1, \lambda) \iff X \sim \text{Exp}(\lambda).$$

Further,

$$X \sim \text{Gamma}\left(\frac{k}{2}, \frac{1}{2}\right) \iff X \sim \chi_k^2.$$

And if

$$Z_1, \dots, Z_k \stackrel{iid}{\sim} N(0, 1),$$

then

$$\sum_{j=1}^k Z_j^2 \sim \chi_k^2.$$

Beta from independent Gamma variables

Let

$$X \sim \text{Gamma}(\alpha, 1), \quad Y \sim \text{Gamma}(\beta, 1),$$

independently.

Then

$$\frac{X}{X + Y} \sim \text{Beta}(\alpha, \beta).$$

So a beta generator can be constructed from two gamma generators.

```
x <- rgamma(10000, shape = 2, rate = 1)
y <- rgamma(10000, shape = 5, rate = 1)
b <- x / (x + y)
c(mean(b), 2 / (2 + 5))
```

```
## [1] 0.2856697 0.2857143
```

Student t from normal and chi-square

Let

$$Z \sim N(0, 1), \quad V \sim \chi_\nu^2,$$

independently. Then

$$T = \frac{Z}{\sqrt{V/\nu}} \sim t_\nu.$$

This explains why the t -distribution has heavier tails: the normal variable is divided by a **random scale**.

F from two chi-square variables

If

$$V_1 \sim \chi_{\nu_1}^2, \quad V_2 \sim \chi_{\nu_2}^2$$

independently, then

$$F = \frac{V_1/\nu_1}{V_2/\nu_2} \sim F_{\nu_1, \nu_2}.$$

Thus normal, chi-square, t , and F distributions form a connected family.

R TASK: build distributions from other distributions

Without using `rt()` or `rf()`:

1. simulate 10 000 observations from t_5 ;
2. simulate 10 000 observations from $F_{5,10}$;
3. compare empirical quantiles with `qt()` and `qf()`;
4. make a Q–Q plot for the simulated t_5 sample;
5. overlay a histogram of the simulated t_5 sample with the theoretical t_5 density.

Solution: build distributions

```
m <- 10000

z <- rnorm(m)
v <- rchisq(m, df = 5)
t_sim <- z / sqrt(v / 5)

v1 <- rchisq(m, df = 5)
v2 <- rchisq(m, df = 10)
f_sim <- (v1 / 5) / (v2 / 10)
```

Solution: build distributions

```
rbind(  
  t_empirical = quantile(t_sim, c(.025, .5, .975)),  
  t_theory = qt(c(.025, .5, .975), df = 5)  
)
```

```
##              2.5%          50%    97.5%  
## t_empirical -2.507817 -0.006971056 2.586567  
## t_theory    -2.570582  0.000000000 2.570582
```

```
rbind(  
  F_empirical = quantile(f_sim, c(.025, .5, .975)),  
  F_theory = qf(c(.025, .5, .975), df1 = 5, df2 = 10)  
)
```

```
##              2.5%          50%    97.5%  
## F_empirical 0.1449714 0.9297785 4.221396  
## F_theory    0.1510767 0.9319332 4.236086
```

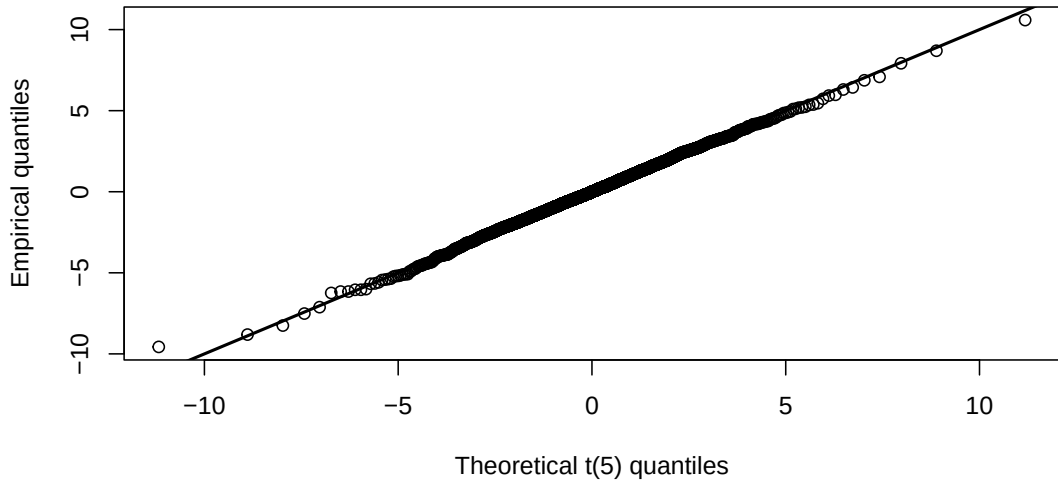
Solution: build distributions

```
p <- ppoints(length(t_sim))
plot(qt(p, df = 5), sort(t_sim),
     xlab = "Theoretical t(5) quantiles",
     ylab = "Empirical quantiles",
     main = "Q-Q plot for simulated t(5)")
abline(0, 1, lwd = 2)

hist(t_sim, probability = TRUE, breaks = 40,
     xlim = c(-6, 6),
     main = "Simulated t(5) sample",
     xlab = "x")
curve(dt(x, df = 5), add = TRUE, lwd = 2)
```

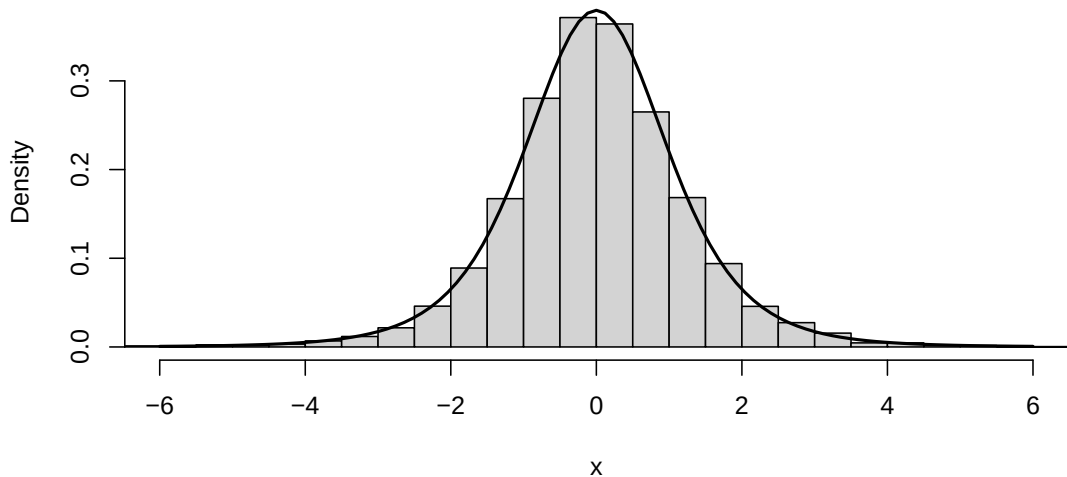
Solution: build distributions

Q-Q plot for simulated $t(5)$



Solution: build distributions

Simulated $t(5)$ sample



Transformation methods

Transformation methods

More generally, suppose

$$X = g(U_1, \dots, U_k)$$

and we know the distribution induced by g .

Then independent uniforms can be transformed into the target distribution.

A particularly important example is the **Box–Muller algorithm** for normal random variables.

Standard bivariate normal

Let $X, Y \stackrel{iid}{\sim} N(0, 1)$. Their joint density is

$$f_{X,Y}(x, y) = \frac{1}{2\pi} \exp \left\{ -\frac{x^2 + y^2}{2} \right\}.$$

Introduce polar coordinates

$$x = r \cos \theta, \quad y = r \sin \theta,$$

with Jacobian

$$\left| \frac{\partial(x, y)}{\partial(r, \theta)} \right| = r.$$

Radius and angle

The joint density becomes

$$f_{R,\Theta}(r, \theta) = \frac{1}{2\pi} r e^{-r^2/2}, \quad r > 0, \quad 0 \leq \theta < 2\pi.$$

This factors as

$$\underbrace{r e^{-r^2/2}}_{f_R(r)} \cdot \underbrace{\frac{1}{2\pi}}_{f_\Theta(\theta)}.$$

Hence R and Θ are independent and

$$\Theta \sim U(0, 2\pi).$$

Distribution of the radius

For

$$S = R^2,$$

we have

$$S = X^2 + Y^2 \sim \chi_2^2.$$

Since

$$\chi_2^2 = \text{Exp}(1/2),$$

we can generate

$$R^2 = -2 \log U_1,$$

and therefore

$$R = \sqrt{-2 \log U_1}.$$

Box–Muller algorithm

Generate independent

$$U_1, U_2 \sim U(0, 1).$$

Then

$$Z_1 = \sqrt{-2 \log U_1} \cos(2\pi U_2),$$

$$Z_2 = \sqrt{-2 \log U_1} \sin(2\pi U_2)$$

are independent $N(0, 1)$ variables.

One pair of uniforms produces **two** normal random variables.

Box-Muller in R

```
rnorm_box_muller <- function(n) {  
  m <- ceiling(n / 2)  
  u1 <- runif(m)  
  u2 <- runif(m)  
  r <- sqrt(-2 * log(u1))  
  theta <- 2 * pi * u2  
  z <- c(r * cos(theta), r * sin(theta))  
  z[seq_len(n)]  
}
```

```
z <- rnorm_box_muller(10000)  
c(mean = mean(z), sd = sd(z))
```

```
##           mean           sd  
## -0.003122925  0.993829887
```

Acceptance-rejection

When inversion is inconvenient

The inverse method may be inconvenient because

- ▶ F^{-1} has no closed form;
- ▶ numerical inversion is expensive;
- ▶ another distribution is much easier to generate.

Acceptance-rejection replaces inversion by proposals from an **instrumental density** g .

Acceptance-rejection theorem

Let f be the target density and g a proposal density.

Assume

$$f(x) \leq c g(x)$$

for all x , for some finite c .

Generate

$$Y \sim g, \quad U \sim U(0, 1),$$

independently, and accept Y when

$$U \leq \frac{f(Y)}{c g(Y)}.$$

Then the accepted value has density f .

Why does it work?

For a proposed value $Y = y$,

$$P(\text{accept} \mid Y = y) = \frac{f(y)}{cg(y)}.$$

Thus the joint contribution of proposals at y that are accepted is

$$g(y) \frac{f(y)}{cg(y)} = \frac{f(y)}{c}.$$

After conditioning on acceptance, the normalizing factor $1/c$ disappears and the density is f .

Acceptance probability

$$\begin{aligned} P(\text{accept}) &= \int P(\text{accept} \mid Y = y) g(y) dy \\ &= \int \frac{f(y)}{cg(y)} g(y) dy \\ &= \frac{1}{c}. \end{aligned}$$

Therefore the expected number of proposals per accepted observation is c .

A good proposal makes c as small as possible.

Choosing the proposal

A useful proposal g should

- ▶ be easy to sample from;
- ▶ be easy to evaluate;
- ▶ cover the support of f ;
- ▶ have tails at least as heavy as the target;
- ▶ resemble the target so that c is small.

If $f(x)/g(x)$ is unbounded, no finite c exists.

Normal target and Cauchy proposal

For

$$f(x) = \phi(x)$$

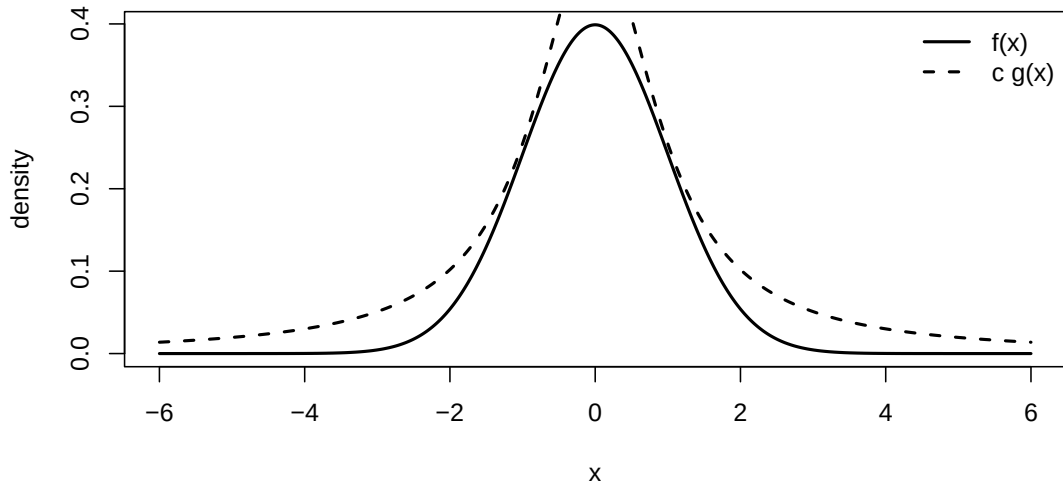
and

$$g(x) = \frac{1}{\pi(1+x^2)},$$

a Cauchy proposal has sufficiently heavy tails.

Normal target and Cauchy proposal

Normal target and Cauchy envelope



Acceptance-rejection in R

```
rnorm_ar <- function(n, c = 1.6) {  
  out <- numeric(n)  
  i <- 0  
  proposals <- 0  
  while (i < n) {  
    y <- rcauchy(1)  
    u <- runif(1)  
    proposals <- proposals + 1  
    if (u <= dnorm(y) / (c * dcauchy(y))) {  
      i <- i + 1  
      out[i] <- y  
    }  
  }  
  attr(out, "acceptance_rate") <- n / proposals  
  out  
}  
z_ar <- rnorm_ar(5000)  
attr(z_ar, "acceptance_rate")
```

```
## [1] 0.6245316
```

Three routes to a normal random variable

We have now seen three conceptually different possibilities:

1. **inverse method**

$$Z = \Phi^{-1}(U);$$

2. **transformation** using Box–Muller;

3. **acceptance-rejection** using a suitable proposal.

Modern implementations may use highly optimized variants, but these three ideas are fundamental.

Random variables and distributions in R

For a distribution named `foo`, R conventionally provides

`dfoo`, `pfoo`, `qfoo`, `rfoo`.

For example:

```
dnorm(0)
```

```
## [1] 0.3989423
```

```
pnorm(1.96)
```

```
## [1] 0.9750021
```

```
qnorm(.975)
```

```
## [1] 1.959964
```

```
rnorm(3)
```

```
## [1] 1.6171309 1.0402518 0.4283162
```

The `r*` functions hide sophisticated generation algorithms behind a simple interface.

Summary

- ▶ Physical randomness and pseudo-randomness solve different problems.
- ▶ Statistical simulation normally favors reproducible PRNGs.
- ▶ Uniform pseudo-random numbers are the basic building blocks.
- ▶ Distributional identities are powerful simulation tools.
- ▶ Inverse transformation gives a general construction.
- ▶ Box–Muller illustrates a transformation method mathematically.
- ▶ Acceptance-rejection works through a dominating proposal density.
- ▶ The quality of the generator and the quality of the transformation both matter.